



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorio de docencia

Laboratorio de Computación Salas A y B

Profesor(a):

Asignatura:

Grupo:

No de práctica(s):

Integrante(s):

No de lista o brigada:

Semestre:

Fecha de entrega:

Observaciones:

Calificación:

Índice

1. Introducción	2
1.1. Planteamiento del Problema	2
1.2. Motivación	2
2. Objetivos	2
2.1. Objetivo General	2
2.2. Objetivos Específicos	3
3. Marco Teórico	3
3.1. Clasificación de Tipos de Datos y Organización en Memoria	3
3.2. Estructuras de Control de Flujo e Operadores de Incremento Abreviado . .	4
3.3. Palabras Reservadas y Definición de Constantes	4
4. Desarrollo	4
4.1. Actividad 1: Evaluación Docente y Control de Aprobados/Reprobados . . .	4
4.2. Actividad 2: Algoritmo de Búsqueda del Mayor	6
4.3. Actividad 3: Tabulación de Progresiones Multiplicativas	7
4.4. Casos de Prueba de Entrada y Salida	8
5. Resultados	9
5.1. Respuesta y Propuesta Algorítmica Generada por la IA	9
5.2. Evidencias de Ejecución en Terminal	9
6. Conclusiones	11
7. Referencias	11

1. Introducción

1.1. Planteamiento del Problema

En la fase inicial del desarrollo de software en Java resulta fundamental comprender la sintaxis básica del lenguaje, la asignación correcta de tipos de datos en la memoria y el flujo de control en programas ejecutados por consola. Para consolidar estos conceptos, en esta práctica se aborda la implementación de tres ejercicios algorítmicos mediante archivos fuente independientes:

1. Diseñar una aplicación en consola para el procesamiento de las calificaciones de un grupo de 10 alumnos. El programa debe solicitar al docente la entrada numérica para cada estudiante, donde el valor 1 representa “aprobado” y el valor 2 indica “reprobado”. Se debe integrar una validación de entrada para garantizar que solo se acepten dichos valores, repetir la solicitud en caso de error, contabilizar ambos totales y desplegar el mensaje “Se bonifica al docente!” únicamente si el número de aprobados es igual o superior a 9.
2. Desarrollar un programa que evalúe una secuencia de 10 números enteros positivos ingresados para encontrar el valor más grande. La solución debe cumplir con la restricción estricta de espacio de declarar únicamente tres variables primitivas dentro del programa (contador, numero, mayor).
3. Elaborar un programa que genere una tabla de progresiones numéricas en la terminal. La aplicación debe calcular $10 * n$, $100 * n$ y $1000 * n$ para valores de n comprendidos secuencialmente entre 1 y 5, utilizando ciclos iterativos y secuencias de escape para alinear los resultados en columnas.

1.2. Motivación

Comprender cómo la JVM administra la memoria al diferenciar entre datos primitivos asignados en la memoria estática (stack) y objetos asignados en la memoria dinámica (heap) es una competencia técnica esencial en la formación académica de la carrera. La resolución de problemas con límites explícitos en el número de variables permite fortalecer el razonamiento algorítmico, fomentando la reutilización de espacios en memoria y la optimización de recursos antes de abordar el paradigma de orientación a objetos y la implementación de mecanismos de validación de entradas y el formato estructurado de salidas en consola mediante caracteres de tabulación aseguran que los programas sean robustos, legibles y prevengan comportamientos inesperados durante su ejecución.

2. Objetivos

2.1. Objetivo General

Crear programas que implementen variables y constantes de diferentes tipos de datos, expresiones y estructuras de control de flujo.

2.2. Objetivos Específicos

- Validar y contabilizar entradas de datos por consola aplicando bucles condicionales anidados e incrementos abreviados.
- Optimizar el uso de recursos de memoria restringiendo el número de variables en algoritmos de comparación condicional.
- Formatear salidas numéricas tabuladas en la interfaz de línea de comandos mediante estructuras repetitivas.

3. Marco Teórico

3.1. Clasificación de Tipos de Datos y Organización en Memoria

El lenguaje Java divide su sistema de tipos en dos categorías principales. Los tipos primitivos (byte, short, int, long, float, double, char, boolean) tienen un tamaño fijo que es idéntico en todas las plataformas, todos tienen su propia clase envoltorio, están integrados de forma invariable y tienen un rango de valores fijo [1]. Estos almacenan directamente el valor asignado y se ubican en la memoria estática (stack). La memoria Stack se caracteriza por una asignación y liberación de espacio acelerada, administrada de forma automática por el hilo de ejecución.

Tipo primitivo	Tamaño	Mínimo	Máximo	Tipo envoltura
boolean	-	-	-	Boolean
char	16 bits	Unicode 0	Unicode + $2^{16} - 1$	Character
byte	8 bits	-128	+127	Byte
short	16 bits	-2^{15}	$+2^{15} - 1$	Short
int	32 bits	-2^{31}	$+2^{31} - 1$	Integer
long	64 bits	-2^{63}	$+2^{63} - 1$	Long
float	32 bits	-3.4×10^{38}	$+3.4 \times 10^{38}$	Float
double	64 bits	-3.4×10^{308}	$+3.4 \times 10^{308}$	Double
void	-	-	-	Void

Por otra parte, los tipos de referencia (como cadenas String, arreglos y clases de la biblioteca estándar o definidas por el usuario) no guardan el objeto en sí dentro del espacio local de la variable, sino una dirección de memoria que apunta hacia la instancia almacenada en la memoria dinámica (heap). Para facilitar la interoperabilidad entre ambos esquemas, Java proporciona clases envoltantes o wrappers (por ejemplo, Integer, Double, Character), permitiendo manipular datos primitivos como objetos cuando las estructuras del lenguaje lo requieren. A diferencia de los primitivos, inician con mayúscula, pueden invocar métodos y admiten el valor null [1]. Sus estructuras principales incluyen:

- String: Secuencia de caracteres delimitada por comillas con métodos de manipulación integrados [1].
- Arreglos (Arrays): Colecciones homogéneas de datos declaradas con corchetes [] [1].

- Clases y Objetos: Las clases actúan como plantillas que definen atributos/métodos, mientras que los objetos son sus instancias en ejecución [1].
- Interfaces: Estructuras abstractas que establecen contratos de comportamiento mediante métodos sin implementar [1].
- Enumeraciones (Enums): Clases especiales para agrupar conjuntos fijos de constantes inmutables [1].

3.2. Estructuras de Control de Flujo e Operadores de Incremento Abreviado

Las estructuras de control determinan la secuencia lógica en la que se ejecutan las instrucciones de un programa. Las estructuras de selección (if, if-else, switch) permiten bifurcar la ejecución en función de la evaluación de expresiones booleanas. Las estructuras repetitivas (for, while, do-while) facilitan la ejecución iterativa de un bloque de código [2]. La sentencia do-while es relevante en procesos de captura de datos debido a que evalúa la condición de paro al final del bloque, garantizando que las instrucciones internas se ejecuten al menos una vez [2, 5].

Para modificar el valor de variables numéricas dentro de estas estructuras iterativas de forma clara, Java ofrece operadores abreviados de incremento (++) y decremento (--), así como operadores de asignación combinada (+=, -=, *=, /=) [3, 5]. El uso de expresiones como contador++ o aprobados += 1 simplifica la escritura del código y mantiene la claridad en el seguimiento de contadores e acumuladores [3, 5].

3.3. Palabras Reservadas y Definición de Constantes

Las palabras reservadas son identificadores predefinidos dentro del compilador javac que poseen un significado sintáctico especial y no pueden ser utilizados como nombres de variables, métodos o clases. Ejemplos clave incluyen modificadores de acceso (public, private), especificadores de contexto (static, void), tipos de datos (int, double) y palabras de control (if, while, return) [4].

Cuando se requiere definir una entidad cuyo valor deba permanecer invariable durante todo el tiempo de ejecución de la aplicación, se utiliza la palabra reservada final. La declaración de constantes previene modificaciones accidentales en memoria, mejora la legibilidad del código y permite al compilador aplicar optimizaciones en el código de bytes (bytecode) [4].

4. Desarrollo

4.1. Actividad 1: Evaluación Docente y Control de Aprobados/Reprobados

Para la resolución de esta actividad, se diseñó e implementó la clase Ejercicio1.java. El algoritmo sigue una secuencia estructurada para garantizar el control de las entradas y el conteo correcto:

1. Inicialización: Se declaran e inicializan las variables enteras aprobados = 0 y reprobados = 0 para llevar el registro del desempeño del grupo.
2. Estructura Principal: Se utiliza un bucle for cuyo índice avanza desde 1 hasta 10, representando el número total de alumnos a evaluar.
3. Validación de Entrada: Dentro del ciclo principal se incluye una estructura do-while para capturar la opción ingresada. Si el usuario introduce un valor distinto de 1 (aprobado) y 2 (reprobado), el programa despliega un mensaje indicando que la selección es inválida y solicita nuevamente la entrada sin avanzar en el conteo de alumnos.
4. Acumulación y Salida: Mediante una estructura if-else, se incrementa la variable aprobados++ cuando la entrada es igual a 1, o reprobados++ cuando es 2. Al finalizar las 10 iteraciones, se imprimen los totales calculados. Finalmente, una sentencia if (aprobados >= 9) evalúa si se cumple el criterio para desplegar el mensaje de bonificación.

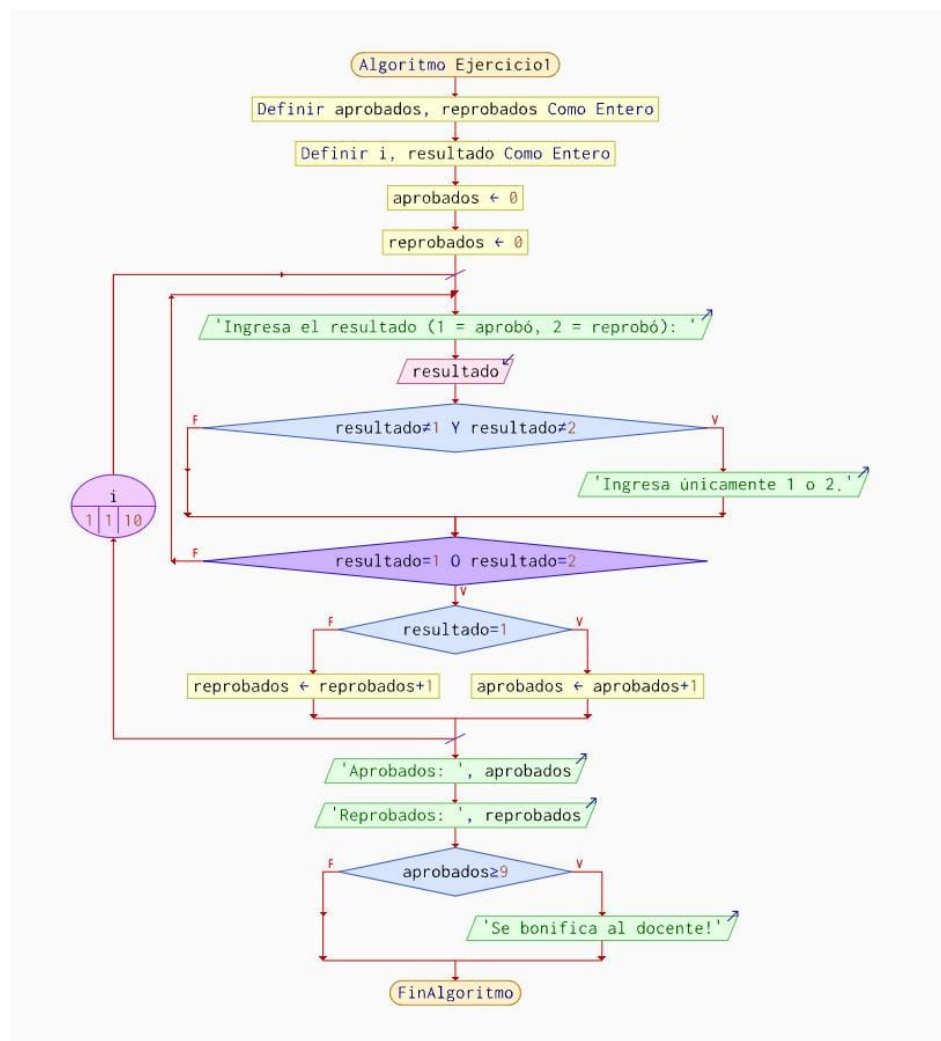


Figura 1: Diagrama de flujo del algoritmo de evaluación docente y conteo de aprobados y reprobados.

4.2. Actividad 2: Algoritmo de Búsqueda del Mayor

El desarrollo del programa Ejercicio2.java se apegó estrictamente a la restricción de emplear únicamente tres variables primitivas de tipo entero: contador, numero y mayor. La lógica algorítmica se ejecutó de la siguiente forma:

1. Declaración de Variables: Se asignan los espacios en memoria estática para “contador” inicializado en 1, “numero” que almacena temporalmente el valor leído en cada turno y “mayor” inicializado en 0 para mantener el registro del valor máximo encontrado.
2. Ciclo de Lectura: Se implementa un bucle while (contador $\leq$ 10) que controla el procesamiento de las 10 entradas numéricas.
3. Evaluación de Condición: En cada iteración se evalúa la expresión if (numero > mayor). De resultar verdadera, el valor de numero sobrescribe el contenido de la variable mayor.
4. Incremento: Se aplica el operador contador++ al final del bloque repetitivo. Una vez concluido el ciclo, se muestra en pantalla el valor almacenado en la variable mayor.

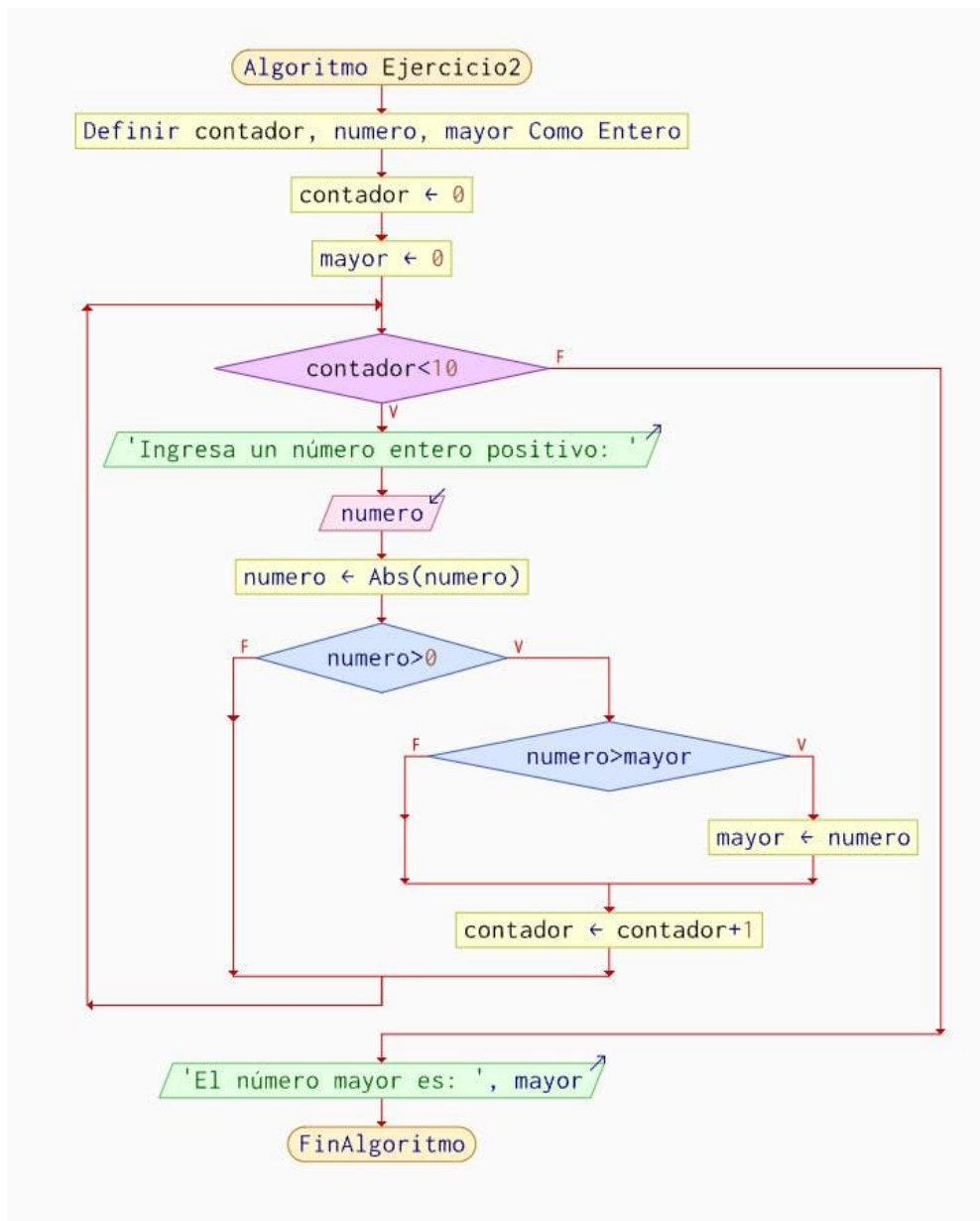


Figura 2: Diagrama de flujo del algoritmo para encontrar el número entero mayor entre diez entradas.

4.3. Actividad 3: Tabulación de Progresiones Multiplicativas

Para generar la tabla de progresiones numéricas en Ejercicio3.java, se utilizó un enfoque algorítmico basado en ciclos e impresiones con formato:

1. Encabezado: Se imprime la línea superior que define los títulos de las columnas: n , $10*n$, $100*n$ y $1000*n$, separando cada etiqueta mediante la secuencia de escape de tabulación `\t`.
2. Generación Iterativa: Se establece un ciclo for con la variable de control n iterando secuencialmente desde 1 hasta 5.
3. Cálculo y Salida: En cada paso del bucle, se evalúan las multiplicaciones corres-

pondientes y se envían a la terminal organizadas con tabuladores, asegurando la alineación vertical de las columnas.

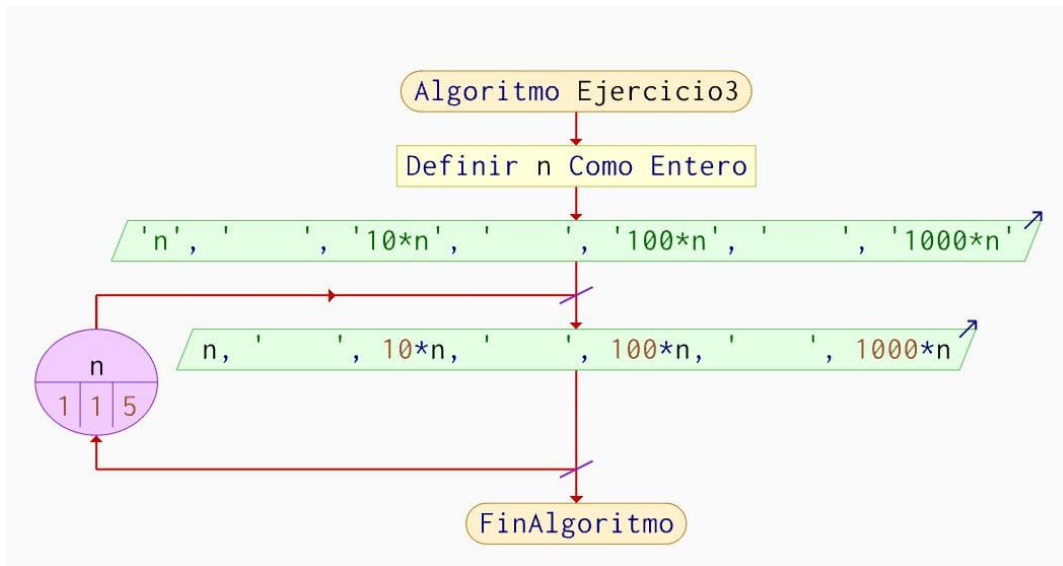


Figura 3: Diagrama de flujo del algoritmo para generar la tabla de progresiones multiplicativas.

4.4. Casos de Prueba de Entrada y Salida

- Prueba 1 (Ejercicio 1 - Evaluación Docente):
 - Caso A: Se ingresan 8 respuestas válidas con valor 1 y 2 respuestas con valor 2. Resultado: Se muestra Aprobados: 8, Reprobados: 2.
 - Caso B: Se ingresan 9 respuestas con valor 1, 1 respuesta con valor 2 y 2 entradas erróneas (5 y 0). Resultado: El programa rechaza las entradas erróneas, solicita de nuevo la información y al concluir muestra Aprobados: 9, Reprobados: 1 junto con el texto “Se bonifica al docente!”.
- Prueba 2 (Ejercicio 2 - Búsqueda del Mayor):
 - Entradas: 14, 52, 8, 93, 27, 61, 85, 40, 19, 73.
 - Resultado: El programa procesa las 10 entradas usando tres variables y despliega “El valor entero más grande es: 93”.
- Prueba 3 (Ejercicio 3 - Tabla de Progresiones):
 - Ejecución: Inicio del programa en consola.
 - Resultado: Impresión de 5 filas numéricas alineadas desde $n=1 \rightarrow (1, 10, 100, 1000)$ hasta $n=5 \rightarrow (5, 50, 500, 5000)$.

Nota: las ejecuciones de estas pruebas se encuentran mas adelante en el apartado de evidencias

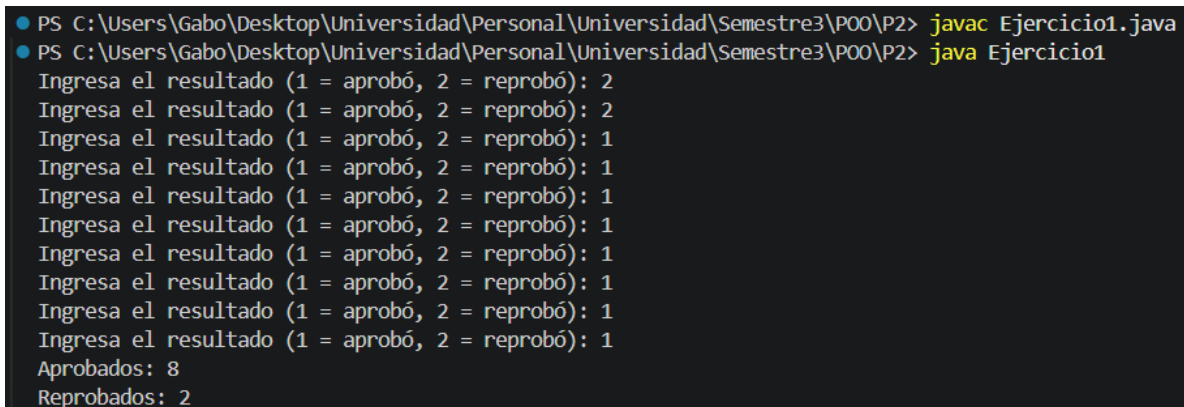
5. Resultados

5.1. Respuesta y Propuesta Algorítmica Generada por la IA

A partir del análisis del código y la estructura lógica propuesta por la herramienta de Inteligencia Artificial para el Ejercicio 1, se destacan los siguientes puntos de evaluación técnica:

- **Estructura de Validaciones:** La propuesta utilizó un bucle do-while interno para filtrar las entradas, asegurando que las selecciones fuera de rango ($\neq 1$ y $\neq 2$) no afecten el conteo de los 10 alumnos.
- **Manejo de Operadores:** Se aplicaron de forma correcta los operadores de incremento abreviado (`aprobados++`, `reprobados++`) y las estructuras de selección condicional (`if-else`) para el manejo eficiente del flujo.
- **Sintaxis y Estilo:** El código respetó las convenciones del lenguaje Java en cuanto a la declaración de variables primitivas, uso de palabras reservadas y estructuración de bloques de código.

5.2. Evidencias de Ejecución en Terminal



```
PS C:\Users\Gabo\Desktop\Universidad\Personal\Universidad\Semestre3\P00\P2> javac Ejercicio1.java
PS C:\Users\Gabo\Desktop\Universidad\Personal\Universidad\Semestre3\P00\P2> java Ejercicio1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 2
Ingresa el resultado (1 = aprobó, 2 = reprobó): 2
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Aprobados: 8
Reprobados: 2
```

Figura 4: Ejecución en terminal del Ejercicio 1, caso A, con 8 aprobados y 2 reprobados.

```

● PS C:\Users\Gabo\Desktop\Universidad\Personal\Universidad\Semestre3\P00\P2> java Ejercicio1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 5
Ingresa únicamente 1 o 2.
Ingresa el resultado (1 = aprobó, 2 = reprobó): 0
Ingresa únicamente 1 o 2.
Ingresa el resultado (1 = aprobó, 2 = reprobó): 2
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Aprobados: 9
Reprobados: 1
Se bonifica al docente!
○ PS C:\Users\Gabo\Desktop\Universidad\Personal\Universidad\Semestre3\P00\P2>

```

Figura 5: Ejecución en terminal del Ejercicio 1, caso B, mostrando el rechazo de entradas inválidas y la bonificación al docente.

```

● PS C:\Users\Gabo\Desktop\Universidad\Personal\Universidad\Semestre3\P00\P2> javac Ejercicio2.java
● PS C:\Users\Gabo\Desktop\Universidad\Personal\Universidad\Semestre3\P00\P2> java Ejercicio2
Ingresa un número entero positivo: 14
Ingresa un número entero positivo: 52
Ingresa un número entero positivo: 8
Ingresa un número entero positivo: 93
Ingresa un número entero positivo: 27
Ingresa un número entero positivo: 61
Ingresa un número entero positivo: 85
Ingresa un número entero positivo: 40
Ingresa un número entero positivo: 19
Ingresa un número entero positivo: 73
El número mayor es: 93
○ PS C:\Users\Gabo\Desktop\Universidad\Personal\Universidad\Semestre3\P00\P2>

```

Figura 6: Ejecución en terminal del Ejercicio 2, con las diez entradas y el valor máximo obtenido.

```

● PS C:\Users\Gabo\Desktop\Universidad\Personal\Universidad\Semestre3\P00\P2> javac Ejercicio3.java
● PS C:\Users\Gabo\Desktop\Universidad\Personal\Universidad\Semestre3\P00\P2> java Ejercicio3
n      10*n      100*n      1000*n
1        10        100       1000
2        20        200       2000
3        30        300       3000
4        40        400       4000
5        50        500       5000
○ PS C:\Users\Gabo\Desktop\Universidad\Personal\Universidad\Semestre3\P00\P2>

```

Figura 7: Ejecución en terminal del Ejercicio 3, mostrando la tabla de progresiones numéricas alineadas.

6. Conclusiones

Los objetivos planteados para esta práctica de laboratorio se cumplieron en su totalidad.

En relación con cada objetivo específico:

1. Se comprobó la efectividad de implementar estructuras de control anidadas como for con do-while e incrementos abreviados en Ejercicio1.java, logrando una validación de entradas que garantiza la integridad de los datos procesados en la consola.
2. Se demostró la viabilidad de optimizar el espacio de memoria en Ejercicio2.java, resolviendo el problema de encontrar el valor máximo mediante la reutilización de únicamente tres variables primitivas en la memoria estática.
3. Se verificó que el uso de ciclos iterativos junto con caracteres de escape de tabulación en Ejercicio3.java permite organizar y estructurar salidas numéricas complejas de forma alineada en la terminal.

La realización de los ejercicios permitió consolidar los conocimientos sobre la sintaxis de Java, la distinción entre tipos de datos, la gestión de la memoria en la JVM y el control lógico del flujo de ejecución en programas de consola.

7. Referencias

Referencias

- [1] IONOS Digital Guide, “Java Data Types: Tipos de datos en Java de un vistazo,” IONOS Digital Guide, 11 de nov. de 2021. [En línea]. Disponible en: <https://www.ionos.mx/digitalguide/paginas-web/desarrollo-web/java-data-types/>
- [2] Oracle, “Control Flow Statements,” The Java Tutorials, 2023. [En línea]. Disponible en: <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/flow.html>
- [3] W3Schools, “Java Assignment Operators,” W3Schools Online Web Tutorials, 2024. [En línea]. Disponible en: https://www.w3schools.com/java/java_operators_assign.asp
- [4] Ciberaula, “Palabras reservadas en Java,” Ciberaula - Portal de Formación en Informática y Programación, 2023. [En línea]. Disponible en: https://www.ciberaula.com/cursos/java/palabras_reservadas_java.php
- [5] C. López, “Operadores de incremento y decremento,” Programación con Java, Facultad de Ingeniería, UNAM, 2022. [En línea]. Disponible en: http://profesores.fib.unam.mx/carlos/java/java_basico2_7.html